
On Defining Neural Averaging

Su Hyeong Lee*
Department of Statistics
University of Chicago

Richard Ngo
Independent

Editors: Marco Fumero, Clementine Domine, Zorah Löhner, Irene Cannistraci, Bo Zhao, Alex Williams

Abstract

What does it even mean to average neural networks? We investigate the problem of synthesizing a single neural network from a collection of pretrained models, each trained on disjoint data shards, using only their final weights and no access to training data. In forming a definition of neural averaging, we take insight from model soup, which appears to aggregate multiple models into a singular model while enhancing generalization performance. In this work, we reinterpret model souping as a special case of a broader framework: Amortized Model Ensembling (AME) for neural averaging, a data-free meta-optimization approach that treats model differences as pseudogradients to guide neural weight updates. We show that this perspective not only recovers model soup but enables more expressive and adaptive ensembling strategies. Empirically, AME produces averaged neural solutions that outperform both individual experts and model soup baselines, especially in out-of-distribution settings. Our results suggest a principled and generalizable notion of data-free model weight aggregation and defines, in one sense, how to perform neural averaging.

1 Introduction

Consider the following problem. Suppose we have N neural networks that are domain experts with differing capabilities, perhaps due to being trained on different shards of a larger data stream. Therefore, each expert is knowledgeable about its own shard, but is largely oblivious or out-of-domain on others. Now, remove all access to the data. We are only given the model weights and nothing else. In this setup, can we synthesize a single model that preserves the knowledge embedded in each expert, using only these final weights and without performing any additional data-driven training?

At a high level, our goal is to construct a form of “average” over these domain experts—a model that aggregates their capabilities and ideally retains their individual performance. But what does it even mean to perform neural averaging? What properties should such an average of neural nets satisfy?

In this work, we propose a framework for answering these questions. We begin by identifying a characteristic for neural network averaging. Neural nets are often highly specialized: they memorize the data they are trained on and tend to generalize less well outside of it [1, 2]. If we succeed in constructing an averaged model, then, even without access to any training data, we expect it to retain the memory of each expert. That is, such a model should perform better on each expert’s original training distribution than it would on a random validation set, despite never having seen any of that data itself.

*Correspondence to: sulee@uchicago.edu

Is such a notion of neural averaging even realizable? A naive approach might be to create a rudimentary mixture model of neural nets: at inference time, sampling from a multinomial distribution over the N experts and using the selected model to make predictions. In expectation, this creates a kind of functional ensemble. However, this strategy is deeply unsatisfying. It requires constantly maintaining access to all N models, which is inefficient, and does nothing to address the fact that each expert is blind to most of the overall data distribution. Deploying any single expert is thus inherently brittle.

A more promising direction comes from the model soup literature [3, 4]. These works show that averaging the weights of multiple independently trained models, e.g., trained with different data orderings or hyperparameters, can yield a new model with stronger generalization. Not only does this suggest that weight-space averaging can be meaningful, but also that it can outperform the individual models being averaged. This empirical success motivates our central question: can we extend model soup to a useful notion of neural averaging?

Our approach is guided by the following intuition. During training, each minibatch gradient update pushes a model through a high-dimensional, non-convex optimization landscape, nudging its weights to encode the statistical features of the data [5, 6]. After enough iterations, the model converges to a local minimum that reflects the distribution it was trained on. We hypothesize that these converged weights implicitly contain enough directional signal that, by computing distance vectors in weight space, we can construct pseudogradients that continue to point toward the expert’s learned minima. If this is true, then we may be able to define a meta-optimization algorithm, operating directly on the expert weights, that mimics the effect of data-driven training. Crucially, this approach would require no actual access to data, relying only on the geometry of the converged models. Moreover, if we employ adaptive optimizers known to accelerate convergence in non-convex settings [7, 8, 9], we may be able to reach high-quality solutions more efficiently [10, 11].

In this paper, we introduce a novel ensembling algorithm that aggregates multiple pretrained neural networks into a single model, without using any data or retraining. Our method constructs updates in weight space by treating model differences as pseudogradients, enabling optimization toward a fused model that integrates the knowledge of each expert. We show that the state-of-the-art model soup strategy [3, 4] emerges as a special case of our framework, corresponding to a single-step gradient descent with a fixed step size. By reframing ensembling as data-free meta-optimization, we offer a new perspective on what it means to “average” in neural network space, that is, to perform neural averaging.

2 Related Literature

2.1 Ensembling Methods in Machine Learning

Traditional ensembling techniques seek to combine multiple models to enhance overall performance, leveraging the strengths of each individual model. Bootstrap Aggregating [12] trains multiple instances of the same model on different subsets of the training data created through random sampling with replacement. This approach can help reduce variance and mitigate overfitting [13, 14]. Boosting [15, 16], on the other hand, builds models sequentially, where each new model attempts to correct the errors of its predecessors. Voting [17] combines the predictions of multiple models by taking a majority vote for classification tasks or averaging for regression tasks. Although these methods are effective in improving model robustness and predictive accuracy, bagging and boosting are expensive to implement for DNN architectures, and voting requires the maintenance of all selected workers in parallel, resulting in an n -fold increase in run-time compute for n ensembled learners during deployment [3]. In contrast, our approach consolidates all n sub-optimal learners into a single, powerful learner.

2.2 Model Ensembling in Deep Learning & Model Soup

The idea of combining multiple neural network weights, either through weighted linear averaging or exponential moving averaging, is a well-studied approach [18, 19, 20, 21, 22, 23, 24]. For instance, Snapshot Ensembling [25] captures multiple local minima in the optimization landscape over a single training run, by encouraging transitory local convergence via a periodic warm-up-ramp-down learning rate schedule. The state dictionary at each minima is saved, and the last m snapshots are linearly

averaged to form a robust predictive model, improving generalization without additional training cost. Additional works have shown that when averaging models that share a portion of the optimization trajectory, including fine-tuning with pre-trained initialization, model accuracies typically do not degrade significantly when averages are taken [26, 27].

A recent development is model soup [3, 22], which showed that in pre-trained transfer learning settings, linear averaging of model parameters (called *soup ingredients*) can make substantial gains on out-of-distribution performance and zero-shot capabilities on downstream tasks. In contrast with previous approaches, souping merges models fine-tuned with hyperparameter configuration diversity (e.g., momentum parameters and learning rates). A recent follow-up work [23] shows that linear interpolation between the weights of fine-tuned models uncovers domains of comparable performance within the simplex whose vertices are the soup ingredients. Despite these empirical observations, it remains unclear why such a linear aggregation of model weights ensembles effectively. In this paper, we show that this procedure can be realized as a very specific sub-category of amortized model ensembling with adaptivity disabled. This entirely optimization-driven approach taken in our work differs significantly from previous interpretations of the superior performance attained via DNN model interpolation, where under strict assumptions, averaging weights discovered over SGD [28] trajectories was viewed as approximately sampling from a Gaussian and computing the maximum likelihood estimator [18, 29].

3 Reformulation of Model Aggregation as Gradient Descent

3.1 Understanding Weight Averaging as a Single Optimization Step

In this section, we show that the linear model averaging process is equivalent to optimizing over an online quadratic loss objective, with the optimum located precisely at each random variable ξ drawn from the population distribution $\mathcal{D}$ of soup ingredients. Therefore, if $\mathcal{D}$ is a single distribution over well-trained models with limited variance, such as over a loss landscape which is empirically well-approximated by a quadratic convex objective, we may intuit that the souped model should display strong performance. In particular, while the optimization of DNNs is a non-convex problem [5], Goodfellow et al [30] observes that loss surfaces can sometimes be approximately convex in practice, over a single trajectory when optimized with SGD. However, if $\mathcal{D}$ is a mixture model comprising multiple distributions associated with well-trained learners situated in disparate regions of the non-convex loss landscape, the simplistic quadratic objective function will exhibit significant fluctuations depending on the sampled distribution. This increased variation implies that non-adaptive gradient descent on a basic quadratic loss will struggle to stabilize the souped output, leading to sub-optimal performance. We now motivate the AME framework by formalizing model soup.

Consider the simple stochastic loss function,

$$f(x) = \frac{1}{2} (\|x - \xi\|^2 - \|\xi\|^2), \quad (1)$$

where $x \in \mathbb{R}^d$ is the high-dimensional space of model parameters and $\xi \sim \mathcal{D}$ for $\mathcal{D}$ the distribution over trained model parameters. After drawing N models $\xi_1, \dots, \xi_N$ to materialize N (online) realizations of the stochastic objective, we perform gradient descent after initializing² at an *arbitrary* x_{pivot} with harmonic learning rate decay $\eta_i = 1/i$, at step i . Letting $x_i = \xi_i$ for notational clarity, the first step of gradient descent gives

$$w_1 = x_{pivot} - \frac{1}{1}(x_{pivot} - x_1),$$

which leads to the second step

$$w_2 = x_1 - \frac{1}{2}(x_1 - x_2) = \frac{1}{2}(x_1 + x_2).$$

Noting the identity

$$\frac{1}{n} \left(\sum_i^n x_i \right) - \frac{1}{n+1} \left(\frac{1}{n} \left(\sum_i^n x_i \right) - x_{n+1} \right) = \frac{1}{n+1} \left(\sum_i^{n+1} x_i \right),$$

²Alternatively, we may initialize at ingredient x_1 and choose the learning rate schedule $\eta_i = 1/(i+1)$.

we obtain after N steps the souped output

$$x_{soup} = w_N = \frac{1}{N} \left(\sum_{i=1}^N x_i \right). \quad (2)$$

Therefore in this paper, we evaluate the utility of regularizing neural nets over the simple objective (1), whose optimum is located at each $\xi \sim \mathcal{D}$. As there exist many distributions such that the empirical average deviates from the maximum likelihood estimator, we seek to utilize stochastic adaptive optimization that effectively supports a diverse range of $\mathcal{D}$ to locate a suitable ensembled model $\xi_{ens} \neq x_{soup}$.

Therefore, we reinterpret model souping as regularization over an online convex objective which is non-adaptively optimized for one epoch in expectation at $\mathbb{E}[\xi]$, the first moment of $\mathcal{D}$. This paves way for the deployment of alternative learning descent algorithms (e.g., Adam [7], Adagrad [8]) for neural averaging, which may provide qualitatively variant results. Intuitively, we posit that the ensemble performance is conditional on the distribution $\mathcal{D}$ of trained model weights. For instance, Lee et al [31] (i.e., Theorem 13) show that under the convex stochastic objective (1), if $\mathcal{D}$ follows a heavy-tailed distribution, then the outputs of non-adaptive gradient descent will display infinite variance as well as incur infinite regret in expectation under any learning rate schedule. Therefore, alternative model updates such as clipped-SGD or adaptive optimizers could be used to enhance robustness and accelerate convergence [11, 32, 33]. We include further detailed discussions along with intuition-building synthetic experiments in Appendix F. In Section 3.2, we formalize these notions into a framework which we call amortized model ensembling. We begin by easing out a precise mathematical formulation, which we generalize to subsume the derivation above.

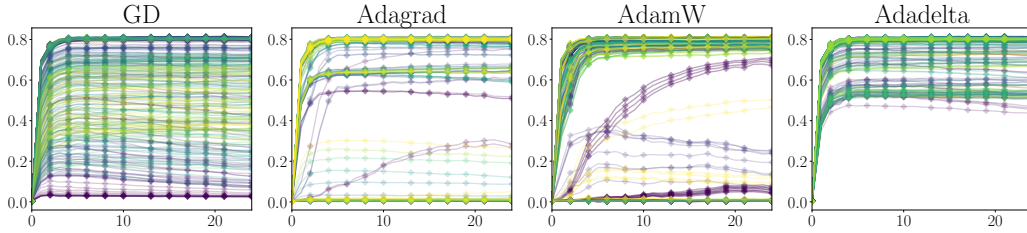


Figure 1: Performance of GD [28], Adagrad [8], AdamW [34], and Adadelata [35] amortized ensembling of ViT-S [36] trained on GLD-23K [37], from left to right, after 1 ensembling epoch. No training nor testing data of any sort are used during the ensemble process (data-free), and only trained model weights are accessed. Plots depict classification accuracy on GLD-23K testing data unseen during training, for a variety of optimizer hyperparameter choices (described in Appendix A). Horizontal axis is the number of model training epochs, different from ensembling epochs, and the color transition indicates the number of ensemble ingredients (models aggregated), ranging from 2 (dark) to 16 (light). This figure shows that on a macroscopic level, each optimizer instantiation induces qualitatively variant ensemble accuracy dynamics, affirming the existence of diverse ensembling strategies uncovered by the AME framework. Details are given in full in Appendix A.2.

3.2 Mathematical Formulation of Amortized Model Ensembling

An alternative way of forming a standard model soup is to use a pivot,

$$x_{soup} = \frac{1}{N} \sum_{i \in [N]} x_i = x_{pivot} - \frac{1}{N} \sum_{i \in [N]} (x_{pivot} - x_i) \quad (3)$$

where $x_1, \dots, x_N$ are the selected model ingredients. Pivoting adds an additional degree of freedom, which allows the formulation of pivoted pseudogradients. We may equivalently interpret (3) as a

sequence of N training steps with initialization x_{pivot} as follows,

$$\begin{aligned} w_1 &= x_{pivot} - \frac{1}{N}(x_{pivot} - x_1), \\ w_2 &= w_1 - \frac{1}{N}(x_{pivot} - x_2), \\ &\vdots \\ w_N &= w_{N-1} - \frac{1}{N}(x_{pivot} - x_N). \end{aligned} \tag{4}$$

Defining $g_i = \zeta_i(x_{pivot} - x_i)/N$ to be the *pivoted pseudogradient* with amplification parameter ζ_i , we formalize this training process as backpropagating with the g_i ,

$$w_i = w_{i-1} - \eta_i g_i. \tag{GD Ensembling}$$

Selecting the fixed learning rate $\eta_i = 1$ and gradient amplification parameter $\zeta_i = 1$ results in standard linear averaging. Therefore, we may consider GD backpropagation variants, such as using a learning rate schedule or applying an adaptive optimizer [7, 8] to the pivoted pseudogradients. For instance, we have for unamplified g_i ,

$$w_i = w_{i-1} - \frac{\eta_i}{\sqrt{\sum_{j=1}^i g_j^2 + \epsilon}} g_i, \tag{Adagrad Ensembling}$$

$$w_i = w_{i-1} - \frac{1}{1 - \beta_1^i} \cdot \frac{\eta_i \hat{m}_i}{(\sqrt{1 - \beta_2^i})^{-1} \sqrt{\hat{v}_i + \epsilon}}, \tag{Adam Ensembling}$$

where for **Adam Ensembling**, the momentum parameters satisfy $\beta_1, \beta_2 \in [0, 1)$ and the moment estimate moving averages are defined as

$$\hat{m}_i = \beta_1 \hat{m}_{i-1} + (1 - \beta_1) g_i, \quad \hat{v}_i = \beta_2 \hat{v}_{i-1} + (1 - \beta_2) g_i^2.$$

Note that any linear interpolation of model weights within the simplex whose vertices are the ensemble ingredients follows as a consequence of **GD Ensembling** with an appropriate learning rate schedule.

Initialization of gradient descent. In standard SGD of modern DNNs, the weight initialization process typically involves a combination of random generation and informative heuristics to control neuron variance during the early stages of training. This is due to the fact that when a model is trained from scratch, we must often start with a very weak, unsophisticated learner that may wander into undesirable landscapes. Future updates gradually realize stronger learners in an online fashion. By contrast, in ensemble settings, we are typically provided with a wealth of fully realized and suitably trained learners, which necessitates the question of the selection of x_{pivot} . Note that x_{pivot} corresponds to the initialization in the gradient descent setting of (4).

Alternative formulation of Amortized Model Ensembling. A non-equivalent formulation of AME is to follow the derivation of Section 3.1. Then, the unamplified i -th pseudogradient is defined as $(w_i - x_i)/N$, which may be amplified using ζ_i or an appropriate learning rate schedule. We further detail this setting under the name *adaptive model pivoting* in Section 3.3. This corresponds to choosing the pivot x_{pivot} adaptively at each step when computing pseudogradients, to reflect the latest and most informative learner. In Section 3.3, we develop a concrete amortized ensembling framework using heuristics that guide our algorithm design. Generally, we take smaller learning steps when the model ingredients are weak, and apply larger, more confident updates when the ingredients are strong, determined autonomously by adaptive optimizers which leverage historical gradient statistics.

3.3 Amortized Model Ensembling

Given unordered model ingredients $\{x_1, \dots, x_N\}$, how should we develop the ensemble? In particular, the step-wise construction of gradient descent (4) requires an ordering of the model ingredients to extract the pivoted pseudogradients. How should we initialize the pivot, and what should guide the choice of ordering? For initialization of the pivot, we propose to select an informative model, such as any of the model ingredients $\{x_1, \dots, x_N\}$ or their standard average (linear soup) as candidates. To order model ingredients, we may construct a list based on any metric such as the validation loss or accuracy on the entire training dataset, from largest to smallest. The ordering can also be task-specific, such as applying a hierarchical total order based on the validation metric evaluated on a particular dataset of interest.

Algorithm 1 Amortized Model Ensembling (Unspecified Optimizer)

Require: Any set of unordered models $\{x_1, \dots, x_N\} \subset \mathbb{R}^d$, pivot $x_{pivot} \in \mathbb{R}^d$
Learning rate schedule η_i , pivoted pseudogradient amplification schedule $\zeta_i, \forall i \in [N]$
1: Construct ordered list $x_1, \dots, x_N$ using metric of choice, initialize $w_0 = x_{pivot}$
2: **for** $i = 1, \dots, N$ **do**
3: Compute $g_i \leftarrow \zeta_i(w - x_i)/N$ for $w = x_{pivot}$ or w_{i-1}
4: $\hat{g}_i \leftarrow \text{Optimizer}(g_i)$
5: Let $w_i \leftarrow w_{i-1} - \eta_i \hat{g}_i$
6: **end for**
7: **return** w_N (ensembled model)

Adaptive model pivoting. We now explore concurrently updating the initial pivot x_{pivot} to the best known value, in order to account for suboptimal choices of x_{pivot} . A version of this can be done by simply letting $x_{pivot} = w_{i-1}$ at step i of model ensembling. For instance, (4) becomes

$$\begin{aligned} w_1 &= x_{pivot} - \frac{1}{N}(x_{pivot} - x_1), \\ w_2 &= w_1 - \frac{1}{N}(w_1 - x_2), \\ &\vdots \\ w_N &= w_{N-1} - \frac{1}{N}(w_{N-1} - x_N), \end{aligned} \tag{5}$$

where pivoted pseudogradients are now defined as $g_i = \zeta_i(w_{i-1} - x_i)/N$. Other choices (e.g., exponentially weighted moving average of w_i) are possible for x_{pivot} to adaptively form pivoted pseudogradients. In particular, we may relate this form (5) back to the intuition provided in Section 1. That is, the pseudogradient is the difference vector between the current state of the ensemble w and a model ingredient x , which takes advantage of the information signals of the training data already encoded in the weight space of x .

There is a vast and active literature on adaptive gradient algorithms, spanning both empirical and theoretical perspectives. Under the AME framework, this literature is now fully accessible to neural network weight-space aggregation, including convergence results and theoretical insights into optimizer selection for models demonstrating a range of characteristics. For more details, we refer to Appendix A.1.

4 Experiment Setups & Empirical Evaluation

General Experiment Setup. We examine the ensemble performance of multiple vision transformers in our evaluation. Per each model, a hyperparameter sweep was conducted whose grid contained a diversity of learning rates and adaptivity/momentum parameters, under different weight decay constants detailed in Appendix A.2. Given any training dataset, the resulting model for each hyperparameter configuration being swept over was saved after every training epoch. During the ensemble stage, all models were hierarchically listed based on their accuracy on a held-out validation set, conditioned on each epoch (e.g., at epoch 20). Afterwards, a select number of top-performing models were ensembled. For robust statistical significance at the expense of computational complexity, we repeated this ensemble procedure, including accessing new model ingredients per epoch, across all training epochs (typically ranging between 0 to 24 or 49). For example, each plot in Figure 2 contains the performance evaluation of approximately 1500 ensembles. We provide full details in Appendix A as well as in figure captions. For all experiments, we set the pseudogradient amplification schedule $\zeta_i = 1$, and employed adaptive pivoting to remain faithful to convex regularization against the stochastic objective (1). The pivots for AME are initialized at the standard model soup.

Figure 2 presents test accuracies of ViT-S ensembles trained on GLD-23K [37]. To assess whether AME constitutes a meaningful form of model averaging, we examine whether the resulting model preserves the memories of its model ingredients. Given that neural networks are known to memorize their training distributions [1, 2], a natural diagnostic is to compare performance on the original

training distributions versus unseen test data—despite AME being applied without access to either. As shown in Appendix B, Figures 5-7 (extensions of Figure 2) and Table 2, the ensembled model consistently achieves higher accuracy on training distributions than on test data, indicating knowledge retention of training data of the ingredients. Moreover, the smoothness of validation curves across a wide range of AME hyperparameters suggests that effective configurations generalize across training regimes: optimizer settings that ensemble well for weakly trained models also transfer to stronger ones.

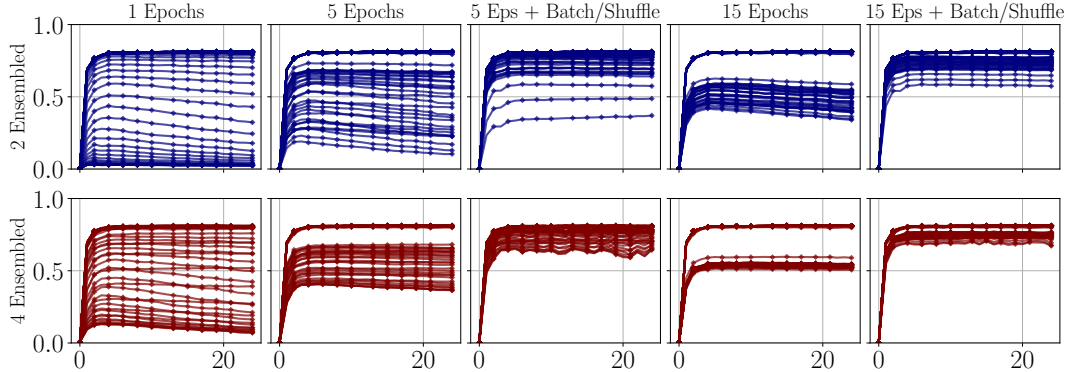


Figure 2: Ensemble test accuracy results of ViT-S fine-tuned on centralized GLD-23K dataset over hyperparameter sweep and experiment setup detailed in Appendix A.2. Each continuous line depicts a single hyperparameter configuration in AME, where an ensemble is formed for each x-axis timestep, which represents training epochs from 0 to 24. The title states the number of ensemble epochs where each model ingredient is treated as a datapoint. AME was instantiated with gradient descent, and batching used two ingredient models per batch. We observe that as the number of ensemble epochs range across 1, 5, and 15, *the ensemble performance improves and solidifies*, manifesting the effects of zero-data model training by simply increasing the number of ensemble epochs. Batching as well as shuffling the neural nets being fused further enhances the ensemble performance. This shows that simply running more meta-optimization epochs, with batching and shuffling of the ensemble ingredients, enhances performance. Additionally, adding more high-performance model ingredients also benefits test accuracy. Additional results are contained in Appendix B.

4.1 Extracting Domain Knowledge via Amortized Model Ensembling

It has previously been observed that balanced linear model averaging (model soup) may empirically strengthen generalization performance. If so, why might souping have this effect? We note that soups correspond to a single epoch of amortized model ensembling instantiated with GD, utilizing a harmonic learning rate schedule (Section 3). In order to capitalize on this phenomenon to design foundational meta-optimization strategies that attain greater generalization capacity, we present an experimental setup to isolate and manifest this behavior, navigated by the following heuristic.

Intuition. During batched backpropagation, each minibatch gradient guides the model across a complex, non-convex optimization landscape, aiming to reach a weight configuration that aligns with the information contained in the minibatch sample. Through repeated updates over multiple batches, the objective is to converge to a local optimum that fits the training data distribution. However, once the model’s weights have converged to such a configuration, the weights already encode the relevant information from the data. By computing a distance vector and adjusting the ensembled weights toward the converged model configurations, could these updates be interpreted as ‘pseudogradients’ toward the training data distribution? If so, this suggests that with suitably trained model weights and an appropriate optimizer, it is possible to achieve an effect analogous to model training while purely ensembling trained weights, without requiring any actual training data whatsoever. Is it possible to localize and prototype this phenomenon?

Corresponding Setup. Motivated by this perspective, we examine the generalization capabilities of amortized ensembles by training ViT-S on CIFAR-10 while consecutively excluding selected classes for any given model. In the first setup, we trained 30 models with optimal AdamW hyperparameters and differing random seeds (impacting data batching and initialization of the classifier head), where the training dataloader singularly and evenly excluded each of the 10 classes. That is, all models

were each trained on 9/10 classes, which forced an entire class of the training dataset (a highly problematic 10%) to be perfectly out-of-domain given any singular model. This definitively capped testing time accuracy at ~ 0.9 . Then, all 30 models were fused, and their performance evaluated. The analogous experiment was repeated at multiple even harsher extremes, where each model was exclusively trained on one to five classes, resulting in 5 sets of 30 models which were 90%, 80%, 70%, 60%, 50% out-of-domain given any singular model. Additional details about the setup are contained in Appendix A.2, and we summarize the final accuracies of the best performing singular model, model soup, and AME in Table 1. A detailed explanation of greedy model soups are presented in Appendix A.4, and supporting plots are provided in Appendix C.

Figure 10 in Appendix C confirms that as more epochs are taken toward the limited-class training data, the pre-trained transformer excessively overfits during fine-tuning, demonstrated by a destabilized test-time loss. However in Figure 11, the performance of well-performing ensembles steadily increase as more training epochs are taken (for a fixed number of ensemble epochs), indicating that each saturated training epoch is materializing a more informative ensemble ingredient in the model weight space. Despite the expectation of gross overfitting to the training classes in this setup, we observe that amortized model ensembling can significantly enhance OOD performance. Surprisingly, even in the case of 90%-OOD models, the best performing ensemble model reached a $\sim 23\%$ classification accuracy on the entire CIFAR-10 dataset. In this setting, each fine-tuned model is able to game the loss by learning a uniformly fixed mapping to the singular training class³. Additionally, Figures 11, 12 show phase transitions in test performance across AME hyperparameter configurations—a phenomenon we flag for future theoretical study (Appendix C).

Table 1: Test-time classification accuracy on CIFAR-10 dataset. For greedy soups, (Desc), (Asc) indicate the ordering of the model ingredients on a held-out validation set (Appendix A.4).

	Best Model Ing.	Soup	Greedy Soup (Asc)	Greedy Soup (Desc)	AdamW Ensemble
10% OOD	87.99%	98.81%	88.66%	89.16%	98.88%
50% OOD	49.65%	96.64%	93.88%	95.19%	97.09%
60% OOD	39.78%	94.88%	74.02%	80.73%	95.46%
70% OOD	29.89%	84.77%	58.44%	66.26%	88.28%
80% OOD	20.00%	61.18%	58.32%	62.91%	66.16%
90% OOD	10.00%	21.76%	22.44%	22.45%	23.22%

Table 2: Train and test accuracies across different OOD percentages

	90% OOD	80% OOD	70% OOD	60% OOD	50% OOD	10% OOD
Best Ingredient Train Accuracy	10.00%	20.00%	29.98%	39.96%	49.95%	89.63%
Best Ingredient Test Accuracy	10.00%	20.00%	29.89%	39.78%	49.65%	87.99%
Best Ensemble Train Accuracy	23.50%	66.85%	89.64%	96.75%	98.58%	99.99%
Best Ensemble Test Accuracy	23.22%	66.16%	88.28%	95.46%	97.09%	98.88%

Discussion. In Section 4.1, we materialize a setting in which ensembling can act to extract specialized domain knowledge, significantly enhancing the generalization capabilities of the ensemble net compared to any singular model or model soup. In Figure 2, we observe that simply employing more AME ensemble epochs helps to enhance, then stabilize the resultant ensemble performance for general hyperparameter selections, revealing the influence of implicit training during the zero-data ensembling process. Furthermore, the performance of the ensemble is higher on the seen training data than on the unseen testing data (e.g., Appendix B or Table 2). Notably, such training-like effects manifest despite AME being completely data-free, where regularization is performed against a convex objective (1). Additionally, instantiating the ensemble using various optimizer strategies can induce qualitatively differing behaviors in even a single ensemble epoch, as cumulatively illustrated in Figure 1. Due to limited space, we examine the effects of batching and shuffling while ensembling a different number of model ingredients in Appendix B. We relegate further discussions to the relevant portions of the Appendix.

4.2 Neural Averaging CIFAR-100

While our evaluations focus on vision classification with ViTs, the principles underlying AME are not tied to specific architectures. Indeed, the method’s independence from training data and its general

³Indeed in Figure 10, the training accuracy achieves 100.00% within the first three epochs.

formulation suggest broad applicability. For instance, what does it mean to perform a neural average over training data? As an illustrative example, consider reinterpreting input images as weights in an augmented architecture: each image is treated as defining the weights of an added initial layer, and a synthetic input consisting entirely of the pixel value 1 is passed through this modified network. All resulting models thus recover the original image classification under the identity data. Therefore, each image corresponds to a unique network, and ensembling the first-layer weights becomes equivalent to ensembling the image data itself.

Motivated by this intuition, we construct a synthetic dataset by sampling 500 image tensors initialized over a uniform distribution $U[0, 1]$, matching the number of training examples per class in CIFAR-100. We then apply class-conditional AME, treating each training image as an ensemble ingredient. The generated pseudo-dataset still elicits correct predictions on the classes the images were synthesized from (e.g., “willow tree (95%)” vs. “maple tree (82%)”) from a typical ViT fine-tuned on CIFAR-100. This suggests that AME captures meaningful structure in weight space that persists even under input synthesis, raising fundamental questions about the nature of neural averaging. We provide additional visualizations in Appendix E.



Figure 3: The first column displays uniformly averaged (souped) images from each CIFAR-100 class’s training set. Subsequent columns show analogous outputs from AME, applied to 500 randomly initialized image tensors per class. Image synthesis was performed using AdamW-based AME with 40 optimization sweeps. The classifier—a ViT fine-tuned solely on the original CIFAR-100 dataset—was never exposed to the synthesized data. Percentages indicate the proportion of synthesized images per class correctly classified by the ViT (out of 500), where random-guessing accuracy is 1%. For further visualizations and AME hyperparameter details, we refer to Appendix E.

4.3 Neural Optimizers as General Algorithms for Online Statistical Estimation

We now propose an alternative interpretation of deep learning optimizers: *optimizers can be viewed as streaming algorithms for computing statistical estimators*. From this perspective, each update involves computing a distance vector between the current estimator and a new data point (or minibatch), adjusted according to rules defined by the optimizer. In AME, we have interpreted these objects as *pseudogradients*, but this principle applies more generally for any type of problem convertible into an online inference setup. This framework naturally accommodates batching and allows us to define an entire family of estimators parameterized by the optimizer’s hyperparameters. In the experiments below, we validate this interpretation empirically and show that AME can outperform the empirical average, especially in heavy-tailed or noisy regimes—linking back to the discussion in Section 3.1. A more in-depth explanation with additional details is given in Appendix F.

Synthetic Experiment Setup. We draw 60,000 samples from a heavy-tailed Cauchy distribution and treat this as the ‘population’ from which observations can be drawn. From this, we subsample

300 points to simulate the ensemble ingredients, analogous to materializing 300 two-dimensional model weights. We then compute both the model soup average and the AME-enssembled result (using Adam). This trial is repeated 300 times, and the results are visualized in Figure 4 (a-b), where (c) provides the result for a standard Gaussian distribution.

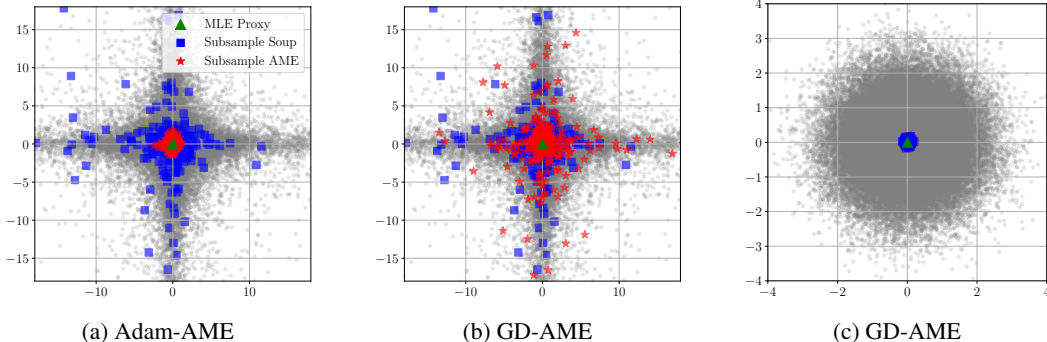


Figure 4: Each panel compares AME ensembles (Adam or GD) against model soup using identical data samples per trial. In (a-b), we verify that under the heavy-tailed Cauchy distribution, Adam-AME demonstrates superior performance than GD-AME. For a non-heavy-tailed Gaussian (c), GD-AME ensembles (red) align closely with the soups (blue) centered around the green MLE, and are therefore visually occluded. Full details and hyperparameter settings are described in Appendix F.

This reveals that model soup exhibits high variance and completely fails to converge toward the proxy MLE under subsampling in heavy-tailed regimes. In contrast, Adam-AME produces estimates that diverge from the empirical average but reliably converge near the proxy MLE. By contrast, GD-AME, which necessarily must only linearly interpolate between the model ingredients, demonstrates similar properties to model soup estimators. This confirms that AME functions as an optimizer-based statistical estimator rather than a simple average, and can yield improved performance depending on the underlying distribution when instantiated with an adaptive optimizer.

The central goal of AME is to harness insights from optimization theory to construct a novel class of empirical estimators that go beyond simple linear averaging. Rather than merely approximating the empirical mean, AME leverages existing optimizers to synthesize ensemble models that encode richer statistical structure—structures often missed by methods such as model soup. This reinterpretation positions adaptive optimizers as numerical routines for computing robust, noise-aware estimators. By initializing at a fixed point and applying pseudogradient updates toward sampled models or datapoints, AME produces solutions that converge closer to the population mean than the empirical average. We believe this perspective motivates the development of future deep neural network optimization-inspired ensembling techniques as statistical estimators in their own right.

5 Conclusion

What does it mean to perform a neural average—to distill a collection of models into a single network that faithfully reflects their collective knowledge? A principled formulation need not rely on access to any data during the averaging process, yet the resulting model should inherit properties of its ingredients, performing better on their respective training distributions than on unseen data despite never observing either. Drawing inspiration from model soup, we develop a notion of neural averaging through the idea of pseudogradients: directions in parameter space that implicitly encode minibatch information. While prior work interprets the out-of-domain gains of model soup through Bayesian lenses (e.g., maximum likelihood estimation), we offer a complementary optimization-based perspective: uniform model averaging is equivalent to a single epoch of gradient descent under a harmonic learning rate schedule. Extending this view, we demonstrate that adding meta-optimization epochs, batching models, and incorporating adaptive optimizers yields consistent performance gains. These insights culminate in the Amortized Model Ensembling (AME) framework, which reframes weight-space averaging as zero-data meta-optimization. Our formulation sheds light on the generalization behavior of weight-averaged deep networks and defines, in one sense, how to perform neural averaging.

References

- [1] Dmitrii Usynin, Moritz Knolle, and Georgios Kaissis. Memorisation in machine learning: A survey of results. *Transactions on Machine Learning Research*, 2024.
- [2] Vitaly Feldman and Chiyuan Zhang. What neural networks memorize and why: Discovering the long tail via influence estimation. *Advances in Neural Information Processing Systems*, 2020.
- [3] Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. *International Conference on Machine Learning*, 2022.
- [4] Charles Dansereau, Milo Sobral, Maninder Bhogal, and Mehdi Zalai. Model soups to increase inference without increasing compute time. *ArXiv*, 2023.
- [5] Anna Choromanska, Mikael Henaff, Michael Mathieu, Gérard Ben Arous, and Yann LeCun. The loss surfaces of multilayer networks. *Proceedings of the 18th International Conference on Artificial Intelligence and Statistics*, 2015.
- [6] Mu Li, Tong Zhang, Yuqiang Chen, and Alexander J. Smola. Efficient mini-batch training for stochastic optimization. *KDD*, 2014.
- [7] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference for Learning Representations*, 2015.
- [8] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12:2121–2159, 2011.
- [9] Sashank Reddi, Satyen Kale, and Sanjiv Kumar. On the convergence of adam and beyond. *International Conference on Learning Representations*, 2018.
- [10] Thanh Huy Nguyen, Umut Simsekli, Mert Gurbuzbalaban, and Gael Richard. First exit time analysis of stochastic gradient descent under heavy-tailed gradient noise. *Advances in Neural Information Processing Systems*, 2019.
- [11] Jingzhao Zhang, Sai Karimireddy, Andreas Veit, Seungyeon Kim, Sashank Reddi, Sanjiv Kumar, and Suvrit Sra. Why are adaptive methods good for attention models? *Advances in Neural Information Processing Systems*, 2020.
- [12] Leo Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, 1996.
- [13] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [14] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The elements of statistical learning: data mining, inference, and prediction*. Springer, 2009.
- [15] Jerome H Friedman. Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001.
- [16] Yoav Freund and Robert E Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.
- [17] Zhi-Hua Zhou. *Ensemble methods: foundations and algorithms*. CRC press, 2012.
- [18] Pavel Izmailov, Dmitrii Podoprikin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. Averaging weights leads to wider optima and better generalization. *Conference on Uncertainty in Artificial Intelligence*, 2018.
- [19] Jean Kaddour, Linqing Liu, Ricardo Silva, and Matt J. Kusner. When do flat minima optimizers work? *Advances in Neural Information Processing Systems*, 2022.
- [20] Timur Garipov, Pavel Izmailov, Dmitrii Podoprikin, Dmitry Vetrov, and Andrew Gordon Wilson. Loss surfaces, mode connectivity, and fast ensembling of dnns. *Advances in Neural Information Processing Systems*, 2018.
- [21] Michael R. Zhang, James Lucas, Geoffrey Hinton, and Jimmy Ba. Lookahead optimizer: k steps forward, 1 step back. *Advances in Neural Information Processing Systems*, 2019.

- [22] Alexandre Ramé, Nino Vieillard, Léonard Hussenot, Robert Dadashi, Geoffrey Cideron, Olivier Bachem, and Johan Ferret. Warm: On the benefits of weight averaged reward models. *International Conference on Machine Learning*, 2024.
- [23] Sara Kangaslahti and David Alvarez-Melis. Continuous language model interpolation for dynamic and controllable text generation. *ArXiv*, 2024.
- [24] Alex Rojas and David Alvarez-Melis. Understanding the role of functional diversity in weight-ensembling with ingredient selection and multidimensional scaling. *ICML Workshop on Foundation Models in the Wild*, 2024.
- [25] Gao Huang, Yixuan Li, Geoff Pleiss, Zhuang Liu, John E. Hopcroft, and Kilian Q. Weinberger. Snapshot ensembles: Train 1, get m for free. *International Conference on Learning Representations*, 2017.
- [26] Behnam Neyshabur, Hanie Sedghi, and Chiyuan Zhang. What is being transferred in transfer learning? *Advances in Neural Information Processing Systems*, 2020.
- [27] Jonathan Frankle, Gintare Karolina Dziugaite, Daniel Roy, and Michael Carbin. Linear mode connectivity and the lottery ticket hypothesis. *International Conference on Machine Learning*, 2020.
- [28] Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.
- [29] Stephan Mandt, Matthew D. Hoffman, and David M. Blei. Stochastic gradient descent as approximate bayesian inference. *Journal of Machine Learning Research*, 18:1–35, 2017.
- [30] Ian J Goodfellow, Oriol Vinyals, and Andrew M Saxe. Qualitatively characterizing neural network optimization problems. *International Conference on Learning Representations*, 2015.
- [31] Su Hyeong Lee, Sidharth Sharma, Manzil Zaheer, and Tian Li. Efficient adaptive federated optimization. *ICML Workshop on Advancing Neural Network Training*, 2024.
- [32] Frederik Kunstner, Robin Yadav, Alan Milligan, Mark Schmidt, and Alberto Bietti. Heavy-tailed class imbalance and why adam outperforms gradient descent on language models. *ArXiv*, 2024.
- [33] Kwangjun Ahn, Xiang Cheng, Minhak Song, Chulhee Yun, Ali Jadbabaie, and Suvrit Sra. Linear attention is (maybe) all you need (to understand transformer optimization). *International Conference on Learning Representations*, 2024.
- [34] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *International Conference on Learning Representations*, 2019.
- [35] Matthew D. Zeiler. Adadelta: An adaptive learning rate method. *ArXiv*, 2012.
- [36] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *International Conference on Learning Representations*, 2021.
- [37] Tobias Weyand, André Araujo, Bingyi Cao, and Jack Sim. Google landmarks dataset v2 - a large-scale benchmark for instance-level recognition and retrieval. In *CVPR*, 2020.
- [38] Umut Simsekli, Levent Sagun, and Mert Gurbuzbalaban. A tail-index analysis of stochastic gradient noise in deep neural networks. *International Conference on Machine Learning*, 2019.
- [39] Umut Simsekli, Lingjiong Zhu, Yee Whye Teh, and Mert Gurbuzbalaban. Fractional underdamped langevin dynamics: Retargeting sgd with momentum under heavy-tailed gradient noise. *International Conference on Machine Learning*, 2020.
- [40] Manzil Zaheer, Sashank Reddi, Devendra Sachan, Satyen Kale, and Sanjiv Kumar. Adaptive methods for nonconvex optimization. *Advances in Neural Information Processing Systems*, 2018.
- [41] Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned stochastic tensor optimization. In *International Conference on Machine Learning*, pages 1842–1850. PMLR, 2018.
- [42] Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost. In *International Conference on Machine Learning*, pages 4596–4604. PMLR, 2018.
- [43] Alex Krizhevsky. Learning multiple layers of features from tiny images. 2009.

- [44] Deyuan Liu, Zecheng Wang, Bingning Wang, Weipeng Chen, Chunshan Li, Zhiying Tu, Dianhui Chu, Bo Li, and Dianbo Sui. Checkpoint merging via bayesian optimization in llm pretraining. *ArXiv*, 2024.
- [45] Suheer Al-Hadhrani, Mohamed El Bachir Menai, Saad Al-Ahmadi, and Ahmad Alnafessah. An effective med-vqa method using a transformer with weights fusion of multiple fine-tuned models. *Applied Sciences*, 13(17), 2023.
- [46] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. *International Conference on Learning Representations*, 2018.
- [47] Timon Gehr, Matthew Mirman, Dana Drachler-Cohen, Petar Tsankov, Swarat Chaudhuri, and Martin Vechev. Ai2: Safety and robustness certification of neural networks with abstract interpretation. *2018 IEEE Symposium on Security and Privacy (SP)*, pages 3–18, 2018.
- [48] Martin Abadi, Andy Chu, Ian Goodfellow, H Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 308–318, 2016.
- [49] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. *International Conference on Artificial Intelligence and Statistics*, 2017.
- [50] Cong Xie, Oluwasanmi Koyejo, Indranil Gupta, and Haibin Lin. Local adaalter: Communication-efficient stochastic gradient descent with adaptive learning rates. *OPT2020: 12th Annual Workshop on Optimization for Machine Learning*, 2020.
- [51] Xiang Li, Kaixuan Huang, Wenhao Yang, Shusen Wang, and Zhihua Zhang. On the convergence of fedavg on non-iid data. *International Conference on Learning Representations*, 2020.
- [52] Jianyu Wang, Qinghua Liu, Hao Liang, Gauri Joshi, and H. Vincent Poor. Tackling the objective inconsistency problem in heterogeneous federated optimization. *Advances in Neural Information Processing Systems*, 2020.
- [53] Yujia Wang, Lu Lin, and Jinghui Chen. Communication-efficient adaptive federated learning. *International Conference on Machine Learning*, 2022.
- [54] Hao Sun, Li Shen, Shixiang Chen, Jingwei Sun, Jing Li, Guangzhong Sun, and Dacheng Tao. Fed-lalr: Client-specific adaptive learning rates achieve linear speedup for non-iid data. *arXiv preprint arXiv:2309.09719*, 2023.
- [55] Su Hyeong Lee, Manzil Zaheer, and Tian Li. Efficient distributed optimization under heavy-tailed noise. *International Conference on Machine Learning*, 2025.
- [56] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith. Federated learning: Challenges, methods, and future directions. *IEEE Signal Processing Magazine*, 37(3):50–60, 2020.
- [57] Arthur Douillard, Qixuan Feng, Andrei A. Rusu, Rachita Chhaparia, Yani Donchev, Adhiguna Kuncoro, Marc’Aurelio Ranzato, Arthur Szlam, and Jiajun Shen. Diloco: Distributed low-communication training of language models. *ICML Workshop on Advancing Neural Network Training*, 2024.
- [58] Sami Jaghouar, Jack Min Ong, and Johannes Hagemann. Opendiloco: An open-source framework for globally distributed low-communication training. *ArXiv*, 2024.
- [59] Bo Liu, Rachita Chhaparia, Arthur Douillard, Satyen Kale, Andrei A. Rusu, Jiajun Shen, Arthur Szlam, and Marc’Aurelio Ranzato. Asynchronous local-sgd training for language modeling. *ICML Workshop on Advancing Neural Network Training*, 2024.
- [60] Sai Praneeth Karimireddy, Martin Jaggi, Satyen Kale, Mehryar Mohri, Sashank Reddi, Sebastian U. Stich, and Ananda Theertha Suresh. Breaking the centralized barrier for cross-device federated learning. *Advances in Neural Information Processing Systems*, 2021.
- [61] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *Arxiv*, 2023.
- [62] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21:1–67, 2020.

- [63] Li Huang, Andrew Shea, Huining Qian, Aditya Masurkar, Hao Deng, and Dianbo Liu. Patient clustering improves efficiency of federated machine learning to predict mortality and hospital stay time using distributed electronic medical records. *Journal of Biomedical Informatics*, 99, 2019.
- [64] Santiago Silva, Boris A. Gutman, Eduardo Romero, Paul M. Thompson, Andre Altmann, and Marco Lorenzi. Federated learning in distributed medical databases: Meta-analysis of large-scale subcortical brain data. *2019 IEEE 16th International Symposium on Biomedical Imaging*, 2019.
- [65] Angela Zhang, Lei Xing, James Zou, and Joseph C. Wu. Shifting machine learning for healthcare from development to deployment and from models to data. *Nature Biomedical Engineering*, 6:1330–1345, 2022.
- [66] Sashank Reddi, Zachary Charles, Manzil Zaheer, Zachary Garrett, Keith Rush, Jakub Konečný, Sanjiv Kumar, and Brendan McMahan. Adaptive federated optimization. *International Conference on Learning Representations*, 2021.
- [67] Suriya Gunasekar, Jason Lee, Daniel Soudry, and Nathan Srebro. Characterizing implicit bias in terms of optimization geometry. *International Conference on Machine Learning*, 2018.
- [68] Bohan Wang, Qi Meng, Wei Chen, and Tie-Yan Liu. The implicit bias for adaptive optimization algorithms on homogeneous neural networks. *International Conference on Machine Learning*, 2021.
- [69] Kazuki Okamura and Yoshiki Otake. Characterizations of the maximum likelihood estimator of the cauchy distribution. *Lobachevskii Journal of Mathematics*, 2022.